

Lecture 01: Spring AI Introduction

What is Spring AI?

- Spring AI is a new application framework for AI engineering.
- It allows developers to integrate Generative AI features directly into Java/Spring Boot applications.
- Works as a layer between your enterprise application and LLMs (Large Language Models).
- Eliminates the need to switch between Java and Python for AI tasks.

Supported AI Providers:

- OpenAI (ChatGPT, GPT-4)
- Anthropic (Claude)
- Google (Gemini)
- Local models via Ollama etc.,

These providers expose Models as a Service (MaaS)—you connect using an API key and credits.

What Spring AI Provides?

AI allows you to integrate Generative AI into your app with ease. It supports:

- Chat Completion – build chatbots/Q&A systems
- Embeddings—convert text into vectors for semantic search
- Text → Image – generate images from prompts
- Audio Transcription – convert speech to text
- Text → Speech – generate speech from text
- Moderation – filter unsafe content

Key Takeaway:

- Spring AI is an AI engineering framework for Java.
- It makes adding AI-powered features like chat, search, image, and audio generation very simple if you already know Spring.

Lecture 02: Why Spring AI

Direct Use of LLMs:

- You can directly use OpenAI or Anthropic SDKs in your Java application.
- **Example with OpenAI:**
 - Create an account, and add credits.
 - Set **OPENAI_API_KEY** as an environment variable.
 - Add the **openai-java dependency** to Maven.
 - Write simple code to send a prompt (e.g., “Tell me a joke”) → get a response.
- This works fine if you only plan to use one model.

The Problem:

- Different providers have different SDKs and code styles.
 - OpenAI code looks one way.
 - Anthropic code looks another way.
- If you want to:
 - Switch from OpenAI → Anthropic → Gemini in the future.
 - Use multiple models in the same project, and you’ll need to rewrite large parts of your code.
- Writing your own abstraction layer is possible but time-consuming and repetitive.

Why Spring AI:

- Spring AI provides abstraction out-of-the-box.
- Instead of writing model-specific code, you use a common API.
- **Benefits:**
 - Cleaner and more maintainable code.
 - Easy to switch providers in the future.
 - **Supports multiple providers:** OpenAI, Anthropic, Microsoft, Amazon, Google, and Ollama.
- Widely adopted because companies prefer abstraction to keep projects flexible and easier to manage.

Key Takeaway:

- Yes, you can connect to models directly.
- But for scalability, flexibility, and maintainability, → Spring AI is the better choice.

Lecture 05: Creating a Spring AI Project

Project Setup with Spring Initializr:

- Go to start.spring.io.
- Choose:
 - **IDE:** IntelliJ IDE
 - **Project Type:** Maven
 - **Language:** Java
 - **Spring Boot Version:** Latest stable
 - **Group ID:** `com.telusko`
 - **Artifact/Project Name:** `springAICode`

Dependencies to Add:

- Spring Web—for building a web application.
- Spring AI (Model-specific):
 - **For OpenAI** → add Spring AI OpenAI Starter.
 - **For Anthropic** → add Anthropic Starter.
 - **For Ollama** → add Ollama Starter.
 - **For Vertex AI** (Google) → add Vertex Starter.
- In this lecture, only Spring Web and Spring AI OpenAI Starter are added.

POM.xml after Adding Dependencies:

- `spring-boot-starter-web`

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

- `spring-ai-openai-spring-boot-starter`

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
```

Running the Project:

- When you run the app, it starts on the Tomcat server.
- But on the first run, it fails.

Why Does It Fail?

- **Error:** OpenAI API key must be set.
- **Missing property:** `spring.ai.openai.api-key=YOUR_KEY_HERE`
- This must be added in **application.properties**.

Note: To fix the error, you need to generate and set the OpenAI API key.

Lecture 06: Create OPEN AI API Key

Why Do We Need an API Key?

- OpenAI services are not free—you need credits to use them.
- An API key is required for Spring AI to connect with OpenAI models.
- Charges are based on tokens (input + output text length).

Steps to Create an API Key

- Go to openai.com and log in.
- Select API Platform (not ChatGPT).
- Open Settings → Billing:
 - Add credits (minimum balance required).
 - **Disable auto-recharge for safety.**
- Go to API Keys → Create New Secret Key:
 - Give it a name (e.g., SpringAI-Demo).
 - Choose a project (default is fine).
 - Assign permissions (simple → all permissions).
 - Click Create → Key generated.
- Copy the key and save it securely.

Add the Key in Project

- Open the **application.properties** file.
- Add the property:
 - **spring.ai.openai.api-key=YOUR_KEY_HERE**

Result:

- **Run** the Spring Boot project again.
- This time, **no API key error**.
- Project starts successfully on **port 8080**.

Lecture 07: Asking Question to OpenAI Model

Setting up the Controller

To connect Spring Boot with OpenAI using Spring AI, we create a controller:

- **Create a new controller class:** `OpenAIController`.
- **Mark it with** `@RestController`.
- **Define a method with** `@GetMapping("/api/{message}")`.
- **Accept user input with** `@PathVariable` `String message`.
- **Response:** return `"Hello World" + message` to confirm setup works.

```
@RestController
public class OpenAIController {

    private OpenAiChatModel chatModel;

    // Constructor Injection
    public OpenAIController(OpenAiChatModel chatModel) {
        this.chatModel = chatModel;
    }

    @GetMapping("/api/{message}")
    public String getAnswer(@PathVariable String message)
    {
        // Send message to OpenAI model
        String response = chatModel.call(message);
        return response;
    }
}
```

Explanation of Code

- **@RestController** → Marks the class as a REST controller.
- **OpenAiChatModel** → Provided by Spring AI to talk to OpenAI.
- **Constructor Injection** → Spring automatically provides the `chatModel` bean.
- **@GetMapping("/api/{message}")** → Exposes an endpoint to accept user input.
- **chatModel.call(message)** → Sends the prompt to OpenAI and gets the response.

How It Works:

- User hits URL: **`http://localhost:8080/api/what is Java?`**
- Controller captures **`"what is Java?"`** as a **`message`**.

- Spring AI sends it to the **OpenAI model**.
- Response returned directly as String.

Output Example:

- **Input 1:** /api/what is Java
- **Response:** "Java is a high-level, object-orientated programming language..."
- **Input 2:** /api/tell me a joke
- **Response:** "Why do programmers prefer dark mode? Because light attracts bugs."

Lecture 08: Working with ChatClient

Why ChatClient?

- Earlier, we worked with **ChatModel**, which lets us interact with LLMs but has limited features.
- **ChatClient** is a higher-level, fluent API built on top of ChatModel.
- Provides more flexibility:
 - Send prompts and receive responses easily.
 - Responses can be **single messages** or **streamed**.
 - Access to more metadata (not just text).

How to Implement:

- **ChatClient is built on ChatModel**
 - You still need a ChatModel instance to create it.
 - Example: `ChatClient.create(chatModel)`.
- **Creating ChatClient**

```
private ChatClient chatClient;

public OpenAIController(OpenAiChatModel chatModel) {
    this.chatClient = ChatClient.create(chatModel);
}
```

- **Using Prompts**
 - Unlike `chatModel.call()`, here we use:
 - `chatClient.prompt(message)` → prepares a request.
 - `.call()` → sends request to LLM.
 - `.content()` → extracts the plain text response.

Advantages:

- Cleaner and more readable API.
- Can fetch not only content but also **chat responses with metadata**.
- Supports structured outputs in the future.

Overall Code:

```
@RestController
public class OpenAIController {
    private ChatClient chatClient;

    public OpenAIController(OpenAiChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @GetMapping("/api/{message}")
    public ResponseEntity<String> getAnswer(@PathVariable String message)
    {
        String response = chatClient.prompt(message)
            .call()
            .content();
        return ResponseEntity.ok(response);
    }
}
```

Key Points:

- **ChatModel vs ChatClient**
 - ChatModel.call("msg") → quick, direct call.
 - ChatClient.prompt("msg").call().content() → more flexible and extendable.
- **ResponseEntity** is used for better practice in Spring and allows status codes along with the response body.
- With ChatClient, you now have more power and flexibility compared to plain ChatModel.

Lecture 09: ChatResponse and Metadata

What We Did Earlier:

- With ChatClient, we used `.prompt(message).call().content()` to get a **string response**.
- That works fine, but what if we need **more than just text**?

About chatResponse:

- Instead of `.content()`, we can use `.chatResponse()`.
- This returns a ChatResponse object, not just plain text.
- Why? Because it contains:
 - The AI response (text).
 - Extra information (metadata) like model name, usage, rate limits, etc.

Fetching Response Text:

```
ChatResponse chatResponse = chatClient.prompt(message)
    .call()
    .chatResponse();

String response = chatResponse
    .getResult()
    .getOutput()
    .getText();
```

- Here, `getResult()` → gives the generation object.
- From that → `getOutput().getText()` → gives the plain text answer.

Fetching Metadata:

```
System.out.println(chatResponse.getMetadata().getModel());
```

- `getMetadata()` returns details such as:
 - Here, the above statement prints 'gpt-4o-mini' in the console.
 - Which **model** is running (e.g., GPT-4o-mini)?
 - **Usage info** (tokens).
 - **Rate limits** and prompt details.
- Useful for **debugging, logging, and monitoring**.

Overall Code:

```
@RestController
public class OpenAIController {
    private ChatClient chatClient;

    public OpenAIController(OpenAiChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @GetMapping("/api/{message}")
    public ResponseEntity<String> getAnswer(@PathVariable String message) {
        ChatResponse chatResponse = chatClient.prompt(message)
            .call()
            .chatResponse();

        // Print model info on console
        System.out.println(chatResponse.getMetadata().getModel());

        // Extract response text
        String response = chatResponse
            .getResult()
            .getOutput()
            .getText();

        return ResponseEntity.ok(response);
    }
}
```

Key Takeaways:

- `.content()` → Quick way to get only text.
- `.chatResponse()` → Full control: response + metadata.
- Metadata helps track **model, usage, rate limits, etc.**
- Good practice: Use `ResponseEntity` for proper HTTP response.
- With `ChatResponse`, you can now handle **both the AI's text** and **extra details** that help in monitoring and debugging.

Lecture 10: ChatClient Builder

Why ChatClient Builder?

- we created ChatClient **programmatically** using:

```
ChatClient.create(chatModel)
```

- But Spring AI also provides a **ChatClient.Builder**.
- Two options:
 - **Programmatic** (ChatClient.create(model)) → good when you want **multiple models**.
 - **Builder** (ChatClient.Builder) works best when you are using **only one model**.

How Builder Works:

- ChatClient.Builder is **auto-configured** by Spring Boot.
- It automatically detects which model (OpenAI, Anthropic, Llama, etc.) is present in your classpath.
- You don't need to pass the ChatModel explicitly.
- Simply inject the builder and call .build().

Implementation:

```
@RestController
public class OpenAIController {
    private ChatClient chatClient;

    // Using Builder instead of manual ChatModel
    public OpenAIController(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    @GetMapping("/api/{message}")
    public ResponseEntity<String> getAnswer(@PathVariable String message) {
        ChatResponse chatResponse = chatClient.prompt(message)
            .call()
            .chatResponse();

        // Print model being used
        System.out.println(chatResponse.getMetadata().getModel());

        // Extract text response
        String response = chatResponse
            .getResult()
            .getOutput()
            .getText();

        return ResponseEntity.ok(response);
    }
}
```

When to Use What:

- **ChatClient.create(model)**
 - Needed when you have multiple models in the same project.
 - You can control which model to use explicitly.
- **ChatClient.Builder**
 - Works when you have only one model.
 - Cleaner and simpler because Spring auto-configures it.

Key Takeaways:

- Builder makes life easier if you're sticking with one AI provider.
- For projects with multiple models → prefer the **programmatic approach**.
- Both ways are valid; choose based on your project setup.
- Use **Builder** for simplicity **when only one model** is in use.

Lecture 11: Spring AI Memory Advisor

Problem with Default ChatClient

- When you ask the model a question, it replies correctly.
- If you then say “more”, the model doesn’t understand—it forgets the earlier context.
- Reason: **LLMs are memoryless by default.** (stateless)
- Tools like ChatGPT appear to remember, but that’s their own wrapper adding memory.

Introducing Advisors in Spring AI

- **Advisors** allow you to intercept or modify requests/responses to the LLM.
- You can use them for:
 - Adding memory to your chat.
 - Safeguarding or censoring respo
 - Monitoring or altering input/output.

Memory with Advisors

- To give your ChatClient memory, use:
 - MessageChatMemoryAdvisor → adds memory capability.
 - InMemoryChatMemory stores conversation in memory.
- This lets the model understand follow-up messages like “more” or “explain in one line”.

Implementation:

```
@RestController
public class OpenAIController {
    private ChatClient chatClient;

    // Create in-memory chat memory
    ChatMemory chatMemory = MessageWindowChatMemory.builder().build();

    // Build ChatClient with memory advisor
    public OpenAIController(ChatClient.Builder builder) {
        this.chatClient = builder
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
            .build();
    }

    @GetMapping("/api/{message}")
    public ResponseEntity<String> getAnswer(@PathVariable String message) {
        ChatResponse chatResponse = chatClient.prompt(message)
            .call()
            .chatResponse();

        String response = chatResponse.getResult().getOutput().getText();
        return ResponseEntity.ok(response);
    }
}
```

Key Takeaways

- **LLMs don't remember past chats** unless you add memory.
- Spring AI provides **Advisors** to extend functionality.
- Using `MessageChatMemoryAdvisor` with `InMemoryChatMemory` allows the app to remember conversations.
- Now follow-up queries like “more” work as expected.

Lecture 12: Running Model Locally with Ollama

Why Run Locally?

- Until now, we used **OpenAI as a service** → requests went to their server.
- Needed an account and credits (paid).
- But you may want to run models **on your own laptop/desktop** for free or customisation.

Ollama – The Local Runner

- Ollama lets you run Large Language Models (LLMs) on your machine.
- Supports many open-source models: Llama, DeepSeek, Phi, Mistral, Gemma, and more.
- Simple setup:
 - Download (~1 GB).
 - Install (Next → Next → Finish → Agree).
 - Open terminal → type 'ollama' to verify.

Working with Models

- Check installed models → ollama list.
- By default, no models are included (each must be downloaded separately).
- Example:
 - **Cmd:** ollama run mistral
- If the model exists, it runs immediately.
- If not, it downloads and installs automatically.

Example Usage

- ollama run mistral → starts the Mistral model.
- Ask: “What is Spring AI?” → may give vague/old answers (since local models are trained on older data).
- Ask: “Tell me a joke.” → responds with programming jokes.
- Follow-up “more” → conversation continues (local memory works).

Things to Know

- **Local models are heavy:**
 - Mistral 7B (~4 GB) → manageable.
 - DeepSeek 32B (~20 GB) → needs high RAM + GPU.
- Start with smaller models.
- Running bigger ones may strain your system.

Integration with Code

- Cloud models (like OpenAI) require API keys.
- **Local Ollama models don't need a key** → they run on your system.
- To connect with your Spring AI project, you'll create a **separate controller** for Ollama integration.
- Current controller is OpenAI-specific.

Key Takeaways

- Ollama = Local LLM runner (free, customizable).
- Supports multiple models, downloaded on demand.
- Local models may be slower and need strong hardware.
- Spring AI will require a new controller setup for Ollama integration.

Lecture 13: Spring AI with Ollama

Why Use Ollama in Spring AI?

- Earlier, we connected Spring AI with **OpenAI**.
- Now, we want to use **Ollama** (local models) inside our Spring AI application.
- Benefit: no API key, no cost, runs directly on your machine.

Steps to Integrate Ollama

- **Create a new Controller**
 - Create the OllamaController class.
 - This controller will use OllamaChatModel.
- **Add Dependency**
 - In pom.xml, add the Ollama dependency from start.spring.io or the official docs.
 - Now you'll have both: OpenAI and Ollama.

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-ollama</artifactId>
</dependency>
```

- **Reload Maven**
 - After updating pom, reload Maven to apply changes.

Implementation:

```
@RestController
public class OllamaController {
    private ChatClient chatClient;

    public OllamaController(OllamaChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @GetMapping("/api/{message}")
    public ResponseEntity<String> getAnswer(@PathVariable String message) {
        ChatResponse chatResponse = chatClient.prompt(message)
            .call()
            .chatResponse();

        System.out.println(chatResponse.getMetadata().getModel());

        String response = chatResponse
            .getResult()
            .getOutput()
            .getText();
        return ResponseEntity.ok(response);
    }
}
```

Key Points

- **Default Model:** If no model is specified, Spring AI with Ollama uses **Mistral**.
- **Missing Models:** If Mistral is not installed, you'll get errors.
- **Custom Model:** Set model via property:
 - `spring.ai.ollama.chat.options.model=deepseek-7b`
- **No Keys Needed:** Local Ollama models don't require API keys.

How It Works

- Run 'ollama list' → check installed models.
- Default model is used unless overridden in properties.
- To switch, install your desired model (e.g., DeepSeek) and update the Spring property.

Takeaways

- Ollama makes Spring AI work with **local open-source LLMs**.
- Add dependency → configure controller → choose model in properties.
- Easy to test: simply **hit /api/{message}** and see responses.
- More control, free usage, but it depends on your machine's hardware.

Lecture 15: Implementing Prompt Template

Why We Need this?

- Earlier, we created a **prompt template** but didn't actually implement it.
- Now, we want to build a **real endpoint** that takes user inputs and dynamically generates prompts.
- Goal: Instead of writing the full prompt every time, pass only values (like type, year, and language) → the template replaces placeholders → AI gives a response.

API Setup

- Create a new **POST API** → /api/recommend.
- Accepts three inputs via @RequestParam:
 - type → movie type (Comedy, Horror, etc.)
 - year → preferred decade/year
 - lang → language preference

Building the Prompt

- Use **PromptTemplate** with placeholders {type}, {year}, and {lang}.
- Example template:

```
I want to watch a {type} movie tonight with good rating,
looking for movies around {year}.
The language I'm looking for is {lang}.
Suggest one specific movie with:
1. Movie Name
2. Basic Plot
3. Cast
4. Length
5. IMDB rating
```

- When API is called, placeholders are replaced with actual values by using the below

```
Prompt prompt = promptTemplate.create(Map.of(
    "type", type,
    "year", year,
    "lang", lang
));
```

Key Flow

- User sends request: POST /api/recommend?type=comedy&year=2010&lang=Telugu

- Controller picks values and creates a **PromptTemplate**.
- Replaces placeholders with real inputs.
- Sends final prompt to AI model using `chatClient.prompt(prompt).call().content()`.
- Returns AI's response in the required format.

Code Implementation:

```
@RestController
public class OpenAIController {
    private ChatClient chatClient;

    public OpenAIController(OpenAiChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @PostMapping("/api/recommend")
    public String recommend(@RequestParam String type, @RequestParam String year, @RequestParam String lang) {

        String temp = """
            I want to watch a {type} movie tonight with good rating,
            looking for movies around this year {year}.
            The language im looking for is {lang}.
            Suggest one specific movie and tell me the cast and length of the movie.

            response format should be:
            1. Movie Name
            2. basic plot
            3. cast
            4. length
            5. IMDB rating
            """;

        PromptTemplate promptTemplate = new PromptTemplate(temp);

        Prompt prompt = promptTemplate.create(Map.of(
            "type", type,
            "year", year,
            "lang", lang
        ));

        String response = chatClient
            .prompt(prompt)
            .call()
            .content();
        return response;
    }
}
```

Key Takeaways:

- **PromptTemplate** makes prompts reusable and structured.
- Always map placeholders with actual values (`Map.of()`).
- We can further improve the format by editing the template.

Lecture 17: Embedding using API Client

Introduction

In this lecture, we explore how to create embeddings using an API client instead of directly writing code. Embeddings are numerical vector representations of text (or other inputs) that help AI systems understand relationships and similarities.

Steps to Create Embeddings with API Client

- **API Endpoint:**
 - Send a request to: `https://api.openai.com/v1/embeddings`
- **Authorization:**
 - **Add a header field:** `Authorization: Bearer <API_KEY>`
 - Replace `<API_KEY>` with your valid OpenAI key.
- **Headers**
 - Content type should be JSON: `Content-Type: application/json`
- **Body (JSON)**
 - Specify the model:



```
{  
  "model": "text-embedding-3-large",  
  "input": "dog"  
}
```

- Common Models:
 - `text-embedding-3-small`
 - `text-embedding-3-large`
- **Response**
 - The API will return a large vector (embedding).
 - Example: “dog” gives an array of thousands of dimensions.

Working with Dimensions:

- Default embeddings are large (e.g., 3000+ dimensions).
- You can specify fewer dimensions (like 2) for simple plotting:

```
{
  "model": "text-embedding-3-large",
  "input": "India",
  "dimensions": 2
}
```

- **Example comparisons:**
 - “Laptop” and “Java” are plotted far apart.
 - “India” and “Russia” appear close together, showing semantic similarity.

Key Observations:

- Closer embeddings = more semantic similarity.
- Example: “India” is closer to “Russia” than to “Laptop.”
- Two-dimensional embeddings can show relationships visually, but for accurate AI tasks, higher dimensions are used.

Summary

- Embeddings can be created easily with an API client.
- Steps:
 - Use a POST request to /v1/embeddings.
 - Add **Authorization** with the API key.
 - Specify the model and input in **Body (JSON)**.
- The response gives vector representations that can be used for semantic similarity, clustering, or visualization.

Lecture 18: Embedding Using Spring AI

Definition

Embeddings are vector representations of text that capture meaning and similarity. With Spring AI, embeddings can be generated directly from a Java application.

Implementation Steps:

- **Controller Setup**

- Create a RestController.
- Define `@PostMapping("/api/embedding")`.
- Accept input text using `@RequestParam String text`.

```
@PostMapping("/api/embedding")
public float[] embedding(@RequestParam String text) {
    return embeddingModel.embed(text);
}
```

- **Embedding Model**

- Use the `EmbeddingModel` provided by Spring AI.
- Inject it with `@Autowired`.
- If multiple embeddings exist (OpenAI, Ollama), use `@Qualifier("openAiEmbeddingModel")`.

```
@Autowired
@Qualifier("openAiEmbeddingModel")
private EmbeddingModel embeddingModel;
```

- **Application Properties: Set API key and model**

- `spring.application.name=SpringAICode`
- `spring.ai.openai.api-key=<your-openai-api-key>`
- `spring.ai.openai.embedding.options.model=text-embedding-3-large`

Code Implementation:

```
@RestController
public class OpenAIController {

    private ChatClient chatClient;

    @Autowired
    private EmbeddingModel embeddingModel;

    public OpenAIController(OpenAiChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @PostMapping("/api/embedding")
    public float[] embedding(@RequestParam String text) {
        return embeddingModel.embed(text);
    }
}
```

Working:

- When a request is sent to `/api/embedding?text=Bottle`, Spring AI converts the text into embeddings.
- Returns an array of float values (vector representation).
- The model can be configured (e.g., `text-embedding-3-large`)

Key Takeaways:

- Qualifier resolves conflicts if multiple embedded beans exist.
- Embeddings are high-dimensional vectors (floats).
- Used for similarity search, clustering, and semantic understanding.

Lecture 19: What is Cosine Similarity

Definition:

Cosine similarity is a mathematical method to measure how similar two vectors are by calculating the cosine of the angle between them. It is widely used in vector databases and embeddings to compare semantic similarity.

Concept:

- Embeddings represent words/text as vectors (multi-dimensional values).
- Similar vectors (e.g., happy and joy) are close to each other.
- Opposite vectors are placed far apart.
- Cosine similarity finds how “close” or “far” two vectors are based on their angle.

Formula for Cosine Similarity:

$$\text{cosine similarity} = \frac{A \cdot B}{||A|| \cdot ||B||}$$

- $A \cdot B \rightarrow$ Dot product of vectors A and B.
- $||A||, ||B|| \rightarrow$ Magnitudes of vectors.
- **Range:**
 - 1 $\rightarrow$ Identical direction (high similarity)
 - 0 $\rightarrow$ Orthogonal (no similarity)
 - -1 $\rightarrow$ Opposite direction

How to implement it in code:

- In Python (NumPy) and libraries, cosine similarity is built-in.
- But, in Java:
 - No direct method by default.
 - Can write a reusable function using the dot product and magnitude formula.
- For multi-dimensional vectors, use summation with a loop to calculate dot product and magnitudes.

Lecture 20: Cosine Similarity Implementation

Definition

Cosine similarity measures how close two vectors are by comparing the cosine of the angle between them. It is useful for checking how semantically similar two words or texts are.

Steps for Implementation

- Controller Setup:
 - Create a RestController with a POST API:
- Generate Embeddings
 - Use EmbeddingModel.embed() to convert both texts into embeddings.

```
/api/similarity
```

```
float[] embedding1 = embeddingModel.embed(text1);  
float[] embedding2 = embeddingModel.embed(text2);
```

- Calculate Cosine Similarity steps in Code
 - Compute the dot product of the vectors.
 - Compute norms (magnitude) of both vectors.
 - Divide the dot product by the product of norms.
 - Multiply by 100 if you want percentage similarity.
- Return Result
 - Returns similarity score as double.
 - Example:
 - dog vs cat → ~0.6
 - computer vs laptop → higher similarity
 - computer vs banana → low similarity (~0.2)

Code Implementation:

```
@RestController
public class OpenAIController {

    private ChatClient chatClient;

    @Autowired
    private EmbeddingModel embeddingModel;

    public OpenAIController(OpenAiChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @PostMapping("/api/similarity")
    public double getSimilarity(@RequestParam String text1, @RequestParam String text2) {
        float[] embedding1 = embeddingModel.embed(text1);
        float[] embedding2 = embeddingModel.embed(text2);

        double dotProduct = 0;
        double norm1 = 0;
        double norm2 = 0;

        for (int i = 0; i < embedding1.length; i++) {
            dotProduct += embedding1[i] * embedding2[i];
            norm1 += Math.pow(embedding1[i], 2);
            norm2 += Math.pow(embedding2[i], 2);
        }

        return dotProduct * 100 / (Math.sqrt(norm1) * Math.sqrt(norm2));
    }
}
```

Usage

- Helps in **semantic search**.
- Finds related results even if exact keywords don't match.
- **Example:** Searching for headphones may return AirPods or Bluetooth speakers.

Lecture 21: Vector Database Introduction

Why Do We Need Vector Databases?

- We already know how embeddings help compare words or text.
- Searching with normal SQL is limited: it matches only exact words.
- Example: Searching “wearable” might miss results like “smartwatch” or “T-shirt.”
- What we want instead is **semantic search**—results based on meaning, not just keywords.

Example Scenario

- Dataset: product_details.json (with product title, description, price, category, and features).
- SQL search for "wearable" → returns nothing if the word is missing in text.
- But semantically, products like **T-shirts** or **smartwatches** are wearable.
This gap is solved by embeddings and vector databases.

How Does It Work?

- **Convert user query into embeddings**
 - Input text like "wearable" → embedding vector.
- **Convert product data into embeddings**
 - Large documents are broken into **chunks**.
 - Each chunk is an embedding vector.
- **Store embeddings in a Vector Database.**
- **Perform similarity search**
 - Matches query embeddings with product embeddings.
 - Retrieves results even if words differ, but meaning is close.

VectorStore in Spring AI

- Spring AI provides an interface called **VectorStore**.
- Operations you can do:
 - Add documents (single or list).
 - Delete documents.
 - Run a similarity search.

Available Implementations

- Supports many backends: **Azure, Cassandra, Chroma, Elasticsearch, MariaDB, Neo4j, Oracle, PgVector, Redis, and SAP.**
- For learning/demo → use **SimpleVectorStore** (in-memory, not for production).
- For real projects → use **PgVector** (PostgreSQL extension).

Key Takeaway:

- Vector databases let us search based on **meaning**, not just keywords. Spring AI makes it simple with **VectorStore**. Start with **SimpleVectorStore** for practice, then move to PgVector or others for production.

Lecture 22: Simple Vector Store

Why Do We Need It?

- We already have product data (title, description, price, category, and features).
- To search semantically (not just by exact words), we need to store this data in a **vector database**.
- Each product (or chunk of text) is converted into embeddings and stored in the **VectorStore**.

Steps to Implement

- **Load the data file**
 - File: product_details.txt stored in resources.
 - Use TextReader (from Spring AI) to read file content.
- **Break into Chunks**
 - Large documents must be split into smaller pieces (chunks).
 - Each chunk is embedded and stored in a vector database.
- **Create DataInitializer Class**
 - Marked with @Component.
 - Method initData() runs at startup (@PostConstruct).
 - Reads the file, splits it into chunks, and stores documents in VectorStore.
- **VectorStore Setup**
 - We use **SimpleVectorStore** (in-memory, good for learning).
 - Defined as a bean in AppConfig.
 - Injected wherever needed (e.g., controller, initializer).

Code Implementation:

```
@Component
public class DataInitializer {

    @Autowired
    private VectorStore vectorStore;

    @PostConstruct
    public void initData() {
        TextReader textReader = new TextReader(new ClassPathResource("product_details.txt"));

        // TokenTextSplitter splitter = new TokenTextSplitter();
        TokenTextSplitter splitter = new TokenTextSplitter(500, 30, 20, 500, false);
        List<Document> documents
            = splitter.split(textReader.get());
        vectorStore.add(documents);
    }
}
```

Key Takeaway:

- SimpleVectorStore helps you load documents, split them into embeddings, and store them for **semantic search**. Start with it for practice, then move to real vector DBs like **PgVector** for production.

Lecture 23: Token Text Splitter

Why Do We Need a Vector Store?

- We use a **Vector Store** to perform **semantic search** or **similarity search**.
- Instead of exact keyword matching, semantic search retrieves results based on meaning.
- Vector Store methods:
 - `add()` → add documents.
 - `delete()` → remove documents.
 - `similaritySearch()` → search for relevant results.

Controller Setup:

- Inject `VectorStore` using `@Autowired` (or constructor injection).
- In the controller, call `vectorStore.similaritySearch(text)` to return matching documents.
- Return type: **List of Documents**.

Document Splitting:

- Input text is **split into chunks (tokens/characters)** before being stored.
- Chunk size affects accuracy:
 - Smaller chunks → more precise, but fragmented.
 - Larger chunks → broader context, but may mix irrelevant data.

Running an Example Search:

- **Example:** searching "tea" returned results like water bottles, kettles, and mugs.
- Sometimes irrelevant results appear (AI hallucination).
- Fine-tuning chunk size improves accuracy.

Limiting Search Results:

- Use `SearchRequest.builder()` to customize search:
 - `query(text)` → search query
 - `topK(2)` → limit results (e.g., top 2 documents only)

```
@PostMapping("/api/product")
public List<Document> getProducts(@RequestParam String text) {
    return vectorStore.similaritySearch(
        SearchRequest.builder()
            .query(text)
            .topK(2)
            .build()
    );
}
```

Key Takeaways:

- **Vector Store** is central for similarity/semantic search.
- Always control **chunk size** for better accuracy.
- Use **SearchRequest** for filtering results and limiting output.

Lecture 24: PG Vector Store Intro

Why Not a Simple Vector Store?

- **Simple Vector Store** is good for **learning and experiments** but not ideal for production.
- Real projects need a **persistent and reliable vector database**.
- That's where **PGVector** comes in.

What is PGVector?

- **PGVector** is an **open-source PostgreSQL extension** for handling vector embeddings.
- It allows storing embeddings in Postgres and performing **similarity searches**.
- Built directly on PostgreSQL, making it reliable and easy to integrate if you already use Postgres

Installation & Setup

- PostgreSQL does **not include PGVector by default**.
- You need to install/enable it separately.
- Options to set it up:
 - **Docker Image/Docker Compose is the easiest** approach.
 - Local installation (platform-specific steps for Windows, Linux, and Mac).

Spring AI + PGVector:

- In Spring AI, PGVector works as a **Vector Store implementation**.
- You will need:
 - **Dependencies** → add the pgvector dependency along with Spring AI.
 - **Database Configuration** → URL, username, password.
 - **Vector Store Properties** → specify:
 - index type
 - distance type (cosine, L2, etc.)
 - dimensions
 - max document batch size

Key Takeaways:

- PGVector lets you **store and query embeddings inside PostgreSQL**.
- Best to use in production where persistence and scalability matter.

Lecture 25: PG Vector Store Configuration

Why Docker?

- We use **Docker Compose** to set up **PGVector** quickly.
- Running PGVector in a container saves us from manual installation.
- Command to start: **docker compose up -d**
- This launches a PostgreSQL instance with PGVector support.

Docker Compose File:

- Define the PGVector service in docker-compose.yml:

```
services:
  pgvector:
    image: 'pgvector/pgvector:pg16'
    environment:
      - 'POSTGRES_DB=telusko'
      - 'POSTGRES_PASSWORD=1234'
      - 'POSTGRES_USER=telusko'
    labels:
      - "org.springframework.boot.service-connection=postgres"
    ports:
      - '5432'
```

- **Database Name:** telusko
- **User:** telusko
- **Password:** 1234
- **Port:** 5432
- Once started, verify that the container is running.

Dependencies:

- Apart from the base vector store, add the **PGVector starter** in pom.xml.
- This allows Spring AI to integrate with PGVector

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-advisors-vector-store</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>
```

Application Properties:

- Configure database connection in application.properties:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/telusko
spring.datasource.username=postgres
spring.datasource.password=1234
spring.jpa.show-sql=true
spring.datasource.driver-class-name=org.postgresql.Driver
```

Key Takeaways:

- PGVector setup requires:
 - **Docker Compose** for containerized PGVector.
 - **Maven dependency** for Spring AI PGVector starter.
 - **Database configuration** in application.properties

Lecture 26: PG Vector Store Implementation

Why Changes Are Needed

- Until now, we used **SimpleVectorStore** (in-memory).
- To use a **real database** for embeddings, we switch to **PGVectorStore** (works with PostgreSQL).
- This requires **extra dependencies** and **application property changes**.

Dependencies to Add

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-pgvector</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-docker-compose</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

Key Code Change

- Replace **SimpleVectorStore** with **PGVectorStore**.
- PGVector needs:
 - **EmbeddingModel** (e.g., OpenAI embeddings)
 - **JdbcTemplate** (bridge between Java & Postgres)

```
@Configuration
public class AppConfig {

    @Bean
    public VectorStore vectorStore(JdbcTemplate jdbcTemplate, EmbeddingModel embeddingModel) {
        return PgVectorStore.builder(jdbcTemplate, embeddingModel)
            .dimensions(1536) // embedding dimensions
            .distanceType(COSINE_DISTANCE) // similarity metric
            .indexType(HNSW) // index type
            .build();
    }
}
```

Application Properties Setup:

- allow-bean-definition-overriding → avoids bean conflicts
- schema-locations → points to schema file (schema.sql)
- init.mode=always → ensures schema loads on startup

```
spring.main.allow-bean-definition-overriding=true  
  
spring.sql.init.schema-locations=classpath:init/schema.sql  
spring.sql.init.mode=always
```

Key Takeaways:

- PGVectorStore lets you **persist vectors in PostgreSQL** (instead of memory).
- Needs:
 - Docker container with Postgres + PGVector
 - Correct schema setup
 - Proper dimensions matching embedding model
- Once configured, semantic search works the same as before—but data is stored **persistently** in Postgres.

Lecture 27: Redis Vector Store Config

Why Redis?

- Redis is a popular **in-memory data structure store**.
- Very fast for vector operations.
- Not ideal for permanent databases, but widely used for caching and vector search.
- We will use it as a **Redis Vector Store** with Spring AI.

Required Dependencies:

Add these two dependencies in pom.xml:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-vector-store-redis</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

Docker Setup:

- Run the Redis stack using Docker:
 - Image will run on **port 6379**.
 - Docker pulls the latest Redis image and starts the container.
 - Run the command given in the terminal:

```
docker run -d --name redis-stack -p 6379:6379 -p 8001:8001 redis/redis-stack:latest
```

Application Properties:

```
spring.data.redis.host=localhost
spring.data.redis.port=6379

spring.ai.vectorstore.redis.index-name=product-index
spring.ai.vectorstore.redis.prefix=product
```

- host → Localhost
- port → 6379 (Docker default)
- index-name → Index name used for storing embeddings
- prefix → Key prefix for Redis storage

Key Notes:

- Redis needs **Jedis client** (Spring Data Redis handles pooling).

- Initialize schema by setting property (default = false).
- Configuration is simpler compared to PGVector.
- Vector search logic remains the same; only the **storage backend changes**.

Lecture 28: Redis Vector Store Implementation

Implementation in Java Code:

- **Step 1: Create Connection**
 - Use **JedisPooled** to connect with the Redis server.
 - Pass **host (localhost)** and **port (6379)** while creating the bean

```
@Bean
public JedisPooled jedisPooled() {
    return new JedisPooled("localhost", 6379);
}
```

- **Step 2: Configure Redis Vector Store**
 - Inject **JedisPooled** and **EmbeddingModel**.
 - Use **RedisVectorStore.builder()** for configuration.
 - Set required properties like **indexName** and **initializeSchema**.

```
@Bean
public VectorStore vectorStore(JedisPooled jedisPooled, EmbeddingModel embeddingModel) {
    return RedisVectorStore.builder(jedisPooled, embeddingModel)
        .indexName("product-index")
        .initializeSchema(true) // ensures index schema creation
        .build();
}
```

Key Points:

- JedisPooled handles Redis connectivity.
- VectorStore integrates **Redis with Spring AI embeddings**.
- indexName defines where vectors are stored.
- initializeSchema(true) ensures the index schema is created automatically.

Lecture 29: What is RAG?

Introduction

- **RAG** stands for **Retrieval Augmented Generation**.
- It is used in **Generative AI** to improve responses by adding context.
- RAG combines **LLM (Large Language Models)** with **external knowledge sources**.

Problems with Only Using LLMs

- **Outdated Knowledge**
 - Models are trained on fixed data (often years old).
 - They cannot access the latest information.
- **Hallucination**
 - If the model does not know an answer, it may generate **incorrect or imaginary responses**.
- **Custom Data Limitation**
 - By default, models do not know your **personal documents or product data**.
 - **Example:** A chatbot cannot answer based on a company's product details without extra context.

Role of RAG:

- RAG helps solve these issues by **retrieving relevant context** from your own data and **augmenting the prompt** before sending it to the model.
- This ensures:
 - More **accurate** responses.
 - **Reduced hallucinations**.
 - Ability to answer from **specific documents or datasets**.

How RAG Works

- **User Prompt**
 - A user sends a query (e.g., “Need details regarding the art kit for kids”).
- **Retrieve Context**
 - Documents are **chunked** and converted into **embeddings** (numbers).
 - These embeddings are stored in a **Vector Database**.
 - Relevant chunks are retrieved based on the query.
- **Augment Prompt**
 - The retrieved chunks are added as **context** to the original query.
- **Generate Response**

- The final prompt (user query + context) is sent to the LLM.
- The model gives a **specific and accurate** response.

Example Scenario:

- Without RAG:
 - **Query:** “Need details about art kit for kids”
 - **Response:** General suggestions, not from your product file.
- With RAG:
 - **Query** is combined with product file data.
 - **Response:** “Art supplies kit for kids includes ...” (from your own product details).

Benefits of RAG:

- Keeps answers **relevant and updated**.
- Reduces chances of **hallucination**.
- Makes chatbots **domain-specific** (works with your data).
- Useful for building **knowledge-driven applications** (e.g., product Q&A, support bots).

Summary:

- RAG = **Prompt + Retrieved Context → Better Generation**.
- It bridges the gap between **static LLM knowledge** and **dynamic, domain-specific information**.

Lecture 30: RAG Implementation

Introduction:

- Implement **Retrieval Augmented Generation (RAG)** in Spring AI.
- Ensure that when a user asks a question, the response is not generic but based on **our product details document** stored in the vector store.

Steps in Implementation:

- **Create API Endpoint**
 - Define a POST endpoint. `/api/ask`
 - Accept the query as a **request parameter**.
 - Return the **string response** directly.

```
@PostMapping("/api/ask")
public String getAnswerUsingRag(@RequestParam String query) {
    return chatClient
        .prompt(query)
        .advisors(new QuestionAnswerAdvisor(vectorStore))
        .call()
        .content();
}
```

- **Use QuestionAnswerAdvisor**
 - Instead of sending a query directly to LLM, add an **advisor**.
 - QuestionAnswerAdvisor is linked with the **vector store**.
 - This ensures that responses are generated using **document embeddings**.

How Does it Work?

- User sends a query (e.g., “Need details about Art kit for kids”).
- Query is combined with **relevant chunks from the vector store**.
- Chat model generates an answer based on **both the query and the retrieved context**.
- Result → **Accurate, document-based response**.

Key Points:

- **Vector Store:** Holds embeddings of product documents.
- **QuestionAnswerAdvisor:** Connects query to vector store.
- **Accurate Retrieval:** Prevents hallucinations and keeps responses relevant.
- **Supports Multiple Formats:** Not limited to text; can also work with **PDFs or external APIs**

Lecture 31: OpenAI Image model

Introduction:

- This lecture focuses on **implementing image generation** using the **OpenAI Image Model** in Spring AI.
- A new controller class, **ImageGenController**, is created for handling image requests.

Code Implementation:

```
@RestController
public class ImageGenController {

    private ChatClient chatClient;
    private OpenAiImageModel openAiImageModel;

    public ImageGenController(OpenAiImageModel openAiImageModel, ChatClient.Builder builder) {
        this.openAiImageModel = openAiImageModel;
        this.chatClient = builder.build();
    }
}
```

Key Points:

- **Dependencies**
 - ChatClient: Used for AI interactions.
 - OpenAiImageModel: Used for image generation.
- **Constructor Injection**
 - Accepts OpenAiImageModel and ChatClient.Builder.
 - Builds a ChatClient instance.
 - Initializes openAiImageModel.

Lecture 32: Image Prompt and Response

Introduction

- To generate images using **the OpenAI Image Model**, we use two important classes:
 - **ImagePrompt** → Represents the input (text query).
 - **ImageResponse** → Represents the output (generated image link).

Code Implementation:

```
@RestController
public class ImageGenController {

    private ChatClient chatClient;

    private OpenAiImageModel openAiImageModel;

    public ImageGenController(OpenAiImageModel openAiImageModel, OpenAiChatModel chatModel) {
        this.openAiImageModel = openAiImageModel;
        this.chatClient = ChatClient.create(chatModel);
    }

    @GetMapping("/image/{query}")
    public String genImage(@PathVariable String query) {

        ImagePrompt prompt = new ImagePrompt(query);

        ImageResponse response = openAiImageModel.call(prompt);
        return response.getResult().getOutput().getUrl();

    }
}
```

Key Points:

- **Endpoint Definition**
 - `@GetMapping("/image/{query}")` → Accepts query as a path variable.
- **Create Prompt**
 - `ImagePrompt prompt = new ImagePrompt(query);`
 - Converts the query text into an image prompt.
- **Call Image Model**
 - `openAiImageModel.call(prompt)` → Sends the prompt to the model.
 - Returns an **ImageResponse**.
- **Extract Image Link**
 - From response → `response.getResult().getOutput().getUrl()`
 - Returns the generated image URL as a string.

Summary:

- Using **ImagePrompt** and **ImageResponse**, we can send a text query and receive a link to the generated image. The method provides a simple way to test image generation with OpenAI in Spring AI.

Lecture 33: Image Options

Introduction:

- When generating images using Spring AI's ImagePrompt, we can customize output by passing **image options** along with the query. These options let us control aspects such as quality, size, style, and number of images.

Image Prompt with Options:

- ImagePrompt supports two constructors:
 - One takes just a query (string).
 - Another accepts a query **and** ImageOptions.
- To create options, we use the **Builder pattern**:

```
OpenAiImageOptions.builder()  
    .quality("hd")  
    .height(1024)  
    .width(1024)  
    .style("natural")  
    .build();
```

Why Builder?

- Builder makes it easy to set multiple values step by step.
- build() returns an object of OpenAiImageOptions (implements ImageOptions).
- This allows flexible customization of generated images.

Common Options

- **Quality**
 - Values: "standard" or "hd"
 - Example: .quality("hd")
- **Height & Width**
 - Supported: 1024 x 1024
 - Example: .height(1024).width(1024)
- **Number of Images (n)**
 - For DALL·E 3 → only one image is supported.
- **Style**
 - Values: "vivid," "natural"
 - Example: .style("natural")

Code Implementation:

```
@GetMapping("/image/{query}")
public String genImage(@PathVariable String query) {

    ImagePrompt prompt = new ImagePrompt(query, OpenAiImageOptions.builder()
        .quality("hd")
        .height(1024)
        .width(1024)
        .style("natural")
        .build());

    ImageResponse response = openAiImageModel.call(prompt);
    return response.getResult().getOutput().getUrl();
}
```

Key Notes

- Errors help discover valid option values (e.g., "hd" must be lowercase).
- Only certain values are allowed; unsupported ones cause exceptions.
- Style affects output appearance (e.g., "natural" gives realistic images).
- Use options to fine-tune generated results instead of default settings.

Lecture 34: Describe Image

Introduction

- After learning how to generate an image from a text prompt, the next step is to **give an image to the AI model and let it describe or analyze it.**

What is Describe Image?

- Instead of generating, we **send an existing image** to the AI model.
- The AI can:
 - Describe what is in the image.
 - Answer a specific prompt (e.g., “Tell me the name of the superhero”).

How It Works (Concept)?

- **Send a Request**
 - A POST request is made to the server (e.g., localhost:8080/image).
 - Along with the request, we provide:
 - **Query** → the instruction or question (e.g., “Describe this image”).
 - **File (image)** → the actual image to analyze.
- **Multipart Data**
 - Since we are uploading an image, the request type is **multipart/form-data**.
 - It includes both the text query and the image file.
- **AI Response**
 - The AI model analyzes the uploaded image.
 - It returns a description or answers based on the given query.

Why Use Describe Image?

- Helps in **image understanding** (e.g., object recognition, scene description).
- Can answer specific questions about an image (not just a generic description).
- Useful for real-world scenarios like
 - The tool is particularly useful for identifying objects in photos.
 - Getting details about uploaded product images.
 - Analyzing scenes in applications like accessibility tools.

Lecture 35: Implementing describe Image

Introduction:

- We implement describe Image using Spring AI by sending both a **query** and an **image file** to the AI model.

Method Setup:

- Create a method with **@PostMapping("/image/describe")**.
- Accept two parameters:
 - **Query**: The instruction or question (e.g., "Name the superhero").
 - **MultipartFile**: The uploaded image file.

Using ChatClient:

- We call the AI model using **chatClient.prompt()**.
- **User Input**:
 - Pass the query text.
 - Attach the image file as media using `MimeTypeUtils.IMAGE_JPEG` (or PNG/GIF if required).
- AI analyzes both the text and the image together.
- The **response content** is returned as output.

Code Implementation:

```
@PostMapping("/image/describe")
public String descImage(@RequestParam String query, @RequestParam MultipartFile file) {

    return chatClient.prompt()
        .user(us -> us.text(query)
            .media(MimeTypeUtils.IMAGE_JPEG, file.getResource()))
        .call()
        .content();
}
```

How Does It Work?

- User sends a **POST request** with a query and an image.
- The method passes both to the **ChatClient**.
- AI processes the image and query.
- The model returns a detailed description or answer.

Example Scenario:

- **Query:** “List items in this image.” → **Response:** Bookshelf, plants, decorative items, chair, etc.

Key Notes:

- **MultipartFile** is used for file uploads.
- The `user()` method allows combining **text and media** inputs.
- Supported media types: JPEG, PNG, GIF.
- The AI can analyze and describe different kinds of images.

Lecture 36: Audio Models Introduction

Introduction:

So far, we have worked with **image models**. Now it's time to explore **audio models** in Spring AI. Audio can be used in two ways:

- **Speech to Text:** Convert spoken audio into written text (Transcription).
- **Text to Speech:** Convert written text into spoken audio.

Types of Audio Models in Spring AI:

- **Transcription (Speech → Text)**
 - Also called **Speech-to-Text (STT)**.
 - You provide an audio file, and the model converts it into text.
 - Common use case: **Captions/Subtitles** for videos.
 - Platforms like **YouTube** and **Udemy** generate captions using transcription models.
 - Useful for accessibility, video content, and automated note generation.
- **Text to Speech (Text → Audio)**
 - Converts a given text into spoken voice output
 - Often called **TTS**.
 - You provide a text input, and the model generates an audio response
 - Options are available to change:
 - **Voice type**
 - **Speed of speech**
 - Use case: Audio learning materials, accessibility tools, or interactive assistants.

Providers for Audio Models:

- **OpenAI**
- **Azure OpenAI** is also supported.

Core Models:

- **OpenAI Transcription Model**
 - Accepts audio input.
 - Uses built-in algorithms to generate accurate text output.
 - Best for caption generation, transcripts, and note-taking.
- **OpenAI Audio Speech Model**
 - Accepts text input.
 - Returns audio output in a natural voice.
 - Provides customization (voice type, speed, etc.).

Key Points:

- **Audio Models** extend AI functionality beyond text and images.
- Two main tasks: **Transcription (STT)** and **Text to Speech (TTS)**.
- Provider: **OpenAI** (primary focus).
- Real-world applications include subtitles, podcasts, accessibility tools, and AI assistants.
- Implementation details will be covered in the next lecture.

Lecture 37 & 38: Audio Transcription Model Speech to Text & Speech to Text Part 2

What is Audio Transcription?

- **Definition:** Audio transcription is the process of converting **spoken audio** into **written text** using AI models.
- **Purpose:** Makes audio content searchable, readable, and useful for captions or subtitles.
- **Spring AI** provides a built-in **OpenAI Audio Transcription Model** for this task.

Why Transcription?

- Platforms like **YouTube**, **Udemy**, etc., use this for **auto-generated captions**.
- Improves **accessibility** for learners.
- Useful for **note-taking**, **subtitles**, and **content indexing**.

Controller Setup:

- Create a separate controller for handling **audio requests**.
- Example: `AudioGenController` as a **RestController**.
- **Speech to Text (STT)** → Convert audio to text.

Code Example: Speech to Text:

```
@RestController
public class AudioGenController {

    private OpenAiAudioTranscriptionModel audioModel;

    public AudioGenController(OpenAiAudioTranscriptionModel audioModel) {
        this.audioModel = audioModel;
    }

    @PostMapping("/api/stt")
    public String speechToText(@RequestParam MultipartFile file) {
        return audioModel.call(file.getResource());
    }
}
```

Explanation:

- `MultipartFile file` → Accepts uploaded audio.
- `audioModel.call(file.getResource())` → Sends audio to the model and returns transcribed text.
- Endpoint: **POST /api/stt**.

Fix Errors:

- **Payload Too Large**
 - By default, Spring supports only **1 MB** uploads.
 - Fix: Add in application.properties:

```
spring.servlet.multipart.max-file-size=20MB  
spring.servlet.multipart.max-request-size=20MB
```

Summary:

- **Speech to Text** is implemented using **the OpenAI Audio Transcription Model**.
- Setup involves:
 - A REST Controller.
 - MultipartFile for audio input.
 - audioModel.call() for conversion.
- Real-world use: captions, notes, accessibility, searchable transcripts.

Lecture 37 & 38: Audio Transcription Model Speech to Text & Speech to Text Part 2

What is Audio Transcription?

- **Definition:** Audio transcription is the process of converting **spoken audio** into **written text** using AI models.
- **Purpose:** Makes audio content searchable, readable, and useful for captions or subtitles.
- **Spring AI** provides a built-in **OpenAI Audio Transcription Model** for this task.

Why Transcription?

- Platforms like **YouTube**, **Udemy**, etc., use this for **auto-generated captions**.
- Improves **accessibility** for learners.
- Useful for **note-taking**, **subtitles**, and **content indexing**.

Controller Setup:

- Create a separate controller for handling **audio requests**.
- Example: **AudioGenController** as a **RestController**.
- **Speech to Text (STT)** → Convert audio to text.

Code Example: Speech to Text:

```
@RestController
public class AudioGenController {

    private OpenAiAudioTranscriptionModel audioModel;

    public AudioGenController(OpenAiAudioTranscriptionModel audioModel) {
        this.audioModel = audioModel;
    }

    @PostMapping("/api/stt")
    public String speechToText(@RequestParam MultipartFile file) {
        return audioModel.call(file.getResource());
    }
}
```

Explanation:

- **MultipartFile file** → Accepts uploaded audio.
- **audioModel.call(file.getResource())** → Sends audio to the model and returns transcribed text.
- Endpoint: **POST /api/stt**.

Fix Errors:

- **Payload Too Large**
 - By default, Spring supports only **1 MB** uploads.
 - Fix: Add in application.properties:

```
spring.servlet.multipart.max-file-size=20MB  
spring.servlet.multipart.max-request-size=20MB
```

Summary:

- **Speech to Text** is implemented using **the OpenAI Audio Transcription Model**.
- Setup involves:
 - A REST Controller.
 - MultipartFile for audio input.
 - audioModel.call() for conversion.
- Real-world use: captions, notes, accessibility, searchable transcripts.

Lecture 39: Audio Transcription Options

Introduction:

- When working with **audio transcription**, we sometimes need **extra control**, such as
 - Adding **timestamps** (for subtitles).
 - Changing **the language** of the output text.
 - Choosing a specific **response format**.
- By default, the transcription method accepts only a file (resource).

Audio Transcription Prompt:

- It accepts two things:
 1. **Resource** (audio file).
 2. **Options** (custom settings).
- OpenAI Audio Transcription Options
- Provides configuration using **the builder pattern**.
- Common options include:
 - `language("es")` → Output transcription in Spanish.
 - `responseFormat(SRT)` → Subtitle format with timestamps.
- This allows us to generate transcriptions with additional details, useful for subtitles and multilingual support.

Code Implementation:

```
@RestController
public class AudioGenController {

    private OpenAiAudioTranscriptionModel audioModel;

    public AudioGenController(OpenAiAudioTranscriptionModel audioModel) {
        this.audioModel = audioModel;
    }

    @PostMapping("api/stt")
    public String speechToText(@RequestParam MultipartFile file) {
        OpenAiAudioTranscriptionOptions options = OpenAiAudioTranscriptionOptions.builder()
            .language("es")
            .responseFormat(OpenAiAudioApi.TranscriptResponseFormat.SRT)
            .build();

        AudioTranscriptionPrompt prompt = new AudioTranscriptionPrompt(file.getResource(), options);

        return audioModel.call(prompt)
            .getResult().getOutput();
    }
}
```

Key Points:

- **AudioTranscriptionPrompt** is required when using options.

- Options provide flexibility for **format** (e.g., SRT for subtitles) and **language translation**.
- The call(prompt) method returns an **AudioTranscriptionResponse**, from which the final text is extracted.

Summary:

- **Without options** → transcription works in default plain text.
- **With options** → transcription supports subtitles, multiple languages, and formats.
- This makes the transcription process more powerful for real-world use like **video captions** and **multilingual content**.

Lecture 40: Audio Speech Model TTS (Text to Speech)

Introduction:

- After transcription (Speech → Text), the next is **Text to Speech (TTS)**.
- TTS converts written **text** into **spoken audio**.
- In Spring AI, this is implemented using the **OpenAiAudioSpeechModel**.

Controller Setup:

- A new endpoint is created for TTS.
 - **Mapping:** POST /api/tts
 - **Accepts:** Text as a request parameter.
 - **Returns:** Byte array (speech output).

Code Implementation:

```
@RestController
public class AudioGenController {

    private OpenAiAudioTranscriptionModel audioModel;
    private OpenAiAudioSpeechModel audioSpeechModel;

    public AudioGenController(OpenAiAudioTranscriptionModel audioModel, OpenAiAudioSpeechModel audioSpeechModel) {
        this.audioModel = audioModel;
        this.audioSpeechModel = audioSpeechModel;
    }

    @PostMapping("/api/tts")
    public byte[] tts(@RequestParam String text) {
        return audioSpeechModel.call(text);
    }
}
```

Testing the API:

- Send a **POST request** to <http://localhost:8080/api/tts>.
- Add text as a request parameter (text=Hello World).
- The response will be a **raw byte array**.
- Save/export as .mp3 to play the audio.

Key Points:

- **OpenAiAudioSpeechModel** is used for TTS.
- Output is returned as a **byte array**.
- REST clients like Insomnia return raw data, which can be saved as an audio file.

Summary:

- TTS converts text into speech.
- Implemented via `audioSpeechModel.call(text)`.
- Returns byte array → save/play as, for example, .mp4.

Lecture 41: Audio Speech Options

Introduction:

- By default, the **Text to Speech (TTS)** model converts text into audio with standard settings. To **customize output** (such as speed and voice), we need to pass **options** using a `SpeechPrompt`.
- Key Concepts:
 - `SpeechPrompt` → Takes **text** and **options**
 - `OpenAiAudioSpeechOptions` → Used to configure customization.
 - Options can be built using the **builder pattern**.

Customizable Options:

- **Speed**
 - Controls playback speed of generated audio
 - Range: 0.25f (slowest) → 4.0f (fastest).
 - Example: 1.5f for faster playback.
- **Voice**
 - Multiple voices available.
 - Default: ALLOY.
 - Other options: ECHO, FABLE, ONYX, NOVA, SHIMMER
 - Example: `Voice.NOVA`.

Code Implementation:

```
@RestController
public class AudioGenController {

    private OpenAiAudioTranscriptionModel audioModel;
    private OpenAiAudioSpeechModel audioSpeechModel;

    public AudioGenController(OpenAiAudioTranscriptionModel audioModel, OpenAiAudioSpeechModel audioSpeechModel) {
        this.audioModel = audioModel;
        this.audioSpeechModel = audioSpeechModel;
    }

    @PostMapping("/api/tts")
    public byte[] tts(@RequestParam String text) {

        OpenAiAudioSpeechOptions options = OpenAiAudioSpeechOptions.builder()
            .speed(1.5f)
            .voice(OpenAiAudioApi.SpeechRequest.Voice.NOVA)
            .build();

        SpeechPrompt prompt = new SpeechPrompt(text, options);

        return audioSpeechModel.call(prompt)
            .getResult().getOutput();
    }
}
```

Key Points:

- **SpeechPrompt** allows passing both text and audio options.

- OpenAiAudioSpeechOptions is created using a builder.
- Customization improves the naturalness of speech output.
- Generated result is still returned as a **byte array** (can be saved as .mp3).

Summary:

- Default TTS can be enhanced with **speed** and **voice** options.
- Speed values range from 0.25f to 4.0f.
- Multiple voices are available (ALLOY, ECHO, NOVA, etc.).
- Use SpeechPrompt with OpenAiAudioSpeechOptions to apply customization.
- This completes the Audio Models section: **Speech to Text (STT)** and **Text to Speech (TTS)** with customization.

Lecture 42: Structured Output Converter

Introduction:

- When AI is integrated into applications, the output should not always be plain text.
- Applications often expect **structured data** such as JSON, lists, or objects instead of free text.
- The **Structured Output Converter** in Spring AI helps transform raw AI output into structured formats that applications can directly use.

Why Structured Output?

- Human-like text responses are fine for conversations.
- For AI engineering, applications need **data in a defined structure**.
- Example formats:
 - JSON
 - List
 - Maps
 - Java Beans / Objects
- This ensures smooth communication between your application and the AI model.

Types of Converters:

- **Bean Output Converter**
 - Maps AI output into an existing **Java Bean/Object**.
 - Useful when you already have defined classes with variables.
- **List Output Converter**
 - Converts AI output into a **list of values**.
- **Map Output Converter**
 - Converts AI output into a **key-value map**
- These converters allow different structured outputs depending on application needs.

How it works?

- **Raw Input** → Sent to AI model through **PromptTemplate**.
- **PromptTemplate** → Combines user input with format instructions.
- **ChatModel** → Generates raw text output from AI.
- **StructuredOutputConverter** → Applies chosen converter:
 - BeanOutputConverter
 - ListOutputConverter
 - MapOutputConverter
- **Final Output** → Returned in structured format for the application.

Key Points:

- Structured output converters ensure AI results are **directly usable by applications**.
- Requires **defining the expected format** so the LLM knows what structure to follow.
- Converters available: **Bean, List, and Map**
- Works by aligning **LLM output format** with the **converter logic**.

Summary:

- Structured Output Converter transforms raw text from AI into structured data
- Helps applications work with predictable formats (JSON, lists, beans, maps).
- Spring AI provides ready-made converters like **BeanOutputConverter**, **ListOutputConverter**, and **MapOutputConverter**.
- Ensures better **integration between AI and enterprise applications**.

Lecture 43: List Output Converter

Introduction:

- Normally, AI output is received as a **string**. But in many cases, applications expect structured data like a **list of values**.
- The **List Output Converter** in Spring AI allows us to transform raw text responses into a list format directly.
- This makes it easy to integrate AI results with applications that require structured data instead of plain strings.

Why Use List Output Converter?

- Without a converter, AI returns a plain string (a list formatted as text).
- With the converter, the AI output is directly transformed into a **List<String>**.
- Helps when an application expects a **collection of values**.
- Example: A Movie recommendation system returning a list of movie names.

Code Implementation:

```
@RestController
public class MovieController {
    private ChatClient chatClient;

    public MovieController(OpenAiChatModel chatModel) {
        this.chatClient = ChatClient.create(chatModel);
    }

    @GetMapping("movies")
    public List<String> getMovies(@RequestParam String name) {
        List<String> movies = chatClient.prompt()
            .user(u -> u.text("List top 5 movies of {name}").param("name", name))
            .call()
            .entity(new ListOutputConverter(new DefaultConversionService()));

        return movies;
    }
}
```

Explanation:

- **ChatClient** → Sends a prompt with the actor name
- **Prompt** → List top 5 movies of {name}.
- **AI Response** → Raw string with movie names.
- **ListOutputConverter** → Converts raw string into a structured **List<String>**
- **Result** → The Application receives the list format directly, which is easy to use.

Key Points:

- **ListOutputConverter** is part of Structured Output Converters in Spring AI.
- Converts text into a **List<String>** using **DefaultConversionService**
- Makes AI results directly consumable by applications.
- Ideal for scenarios like **movie lists, product lists, and recommendations**.

Summary:

- AI responses are usually text, but applications may need structured formats
- **ListOutputConverter** converts AI output into **List<String>**.
- **Simple integration:** AI → String → Converter → Structured List.
- Useful for building real-world applications expecting list-based data.

Lecture 44: Bean Output Converter

Introduction:

- Often, AI returns plain text or lists, but many applications expect data in the form of a **Java object (Bean)**.
- The **Bean Output Converter** in Spring AI is used to map AI responses into a structured Java class format.

Why Use a Bean Output Converter?

- Plain string output is not structured.
- Bean Output Converter maps the response into a **Java Bean**.
- Useful when you already have entity classes with defined fields.
- Ensures seamless integration with existing models.

Code Implementation:

- **Movie Class**

```
public class Movie {  
    private String movieName;  
    private String leadActor;  
    private String director;  
    private int year;  
  
    // getters and setters  
}
```

- **Logic to Convert into Bean**

```
@GetMapping("/movie")  
public Movie getMovieData(@RequestParam String name) {  
  
    BeanOutputConverter<Movie> opCon = new BeanOutputConverter<Movie>(Movie.class);  
  
    Movie movie = chatClient.prompt()  
        .user(u -> u.text("Get me the best movie of {name}").param("name", name))  
        .call()  
        .entity(new BeanOutputConverter<Movie>(Movie.class));  
  
    return movie;  
}
```

Explanation:

- **Entity Class** → Defines structure (movieName, leadActor, director, year).
- **BeanOutputConverter<Movie>** → Informs Spring AI to use **the Movie** format.
- **Prompt** → e.g., "Get me the best movie of Tom Hanks."

- **AI Output** → Returns structured JSON.
- **Converter** → Converts JSON into a **Movie** object.
- **Example Output:**

```
{  
  "movieName": "Forrest Gump",  
  "leadActor": "Tom Hanks",  
  "director": "Robert Zemeckis",  
  "year": 1994  
}
```

Key Points:

- Maps AI output directly into a Java Bean.
- Requires specifying the class type.
- Works well when the AI is guided to output in JSON.
- Makes results directly usable in enterprise applications.

Summary:

- **Bean Output Converter** → Converts AI text into Java objects.
- Best for scenarios where structured data is needed
- Requires entity definition and converter setup.
- Ensures clean integration between AI output and business logic.

Lecture 45: Bean Output Converter with List

Introduction:

- The **Bean Output Converter** is not limited to converting a single object
- It can also be used to fetch and map a **list of objects** in JSON format.
- This is useful when we want multiple records, such as a list of movies for a given actor.

Why Use Bean Output Converter with List?

- For a single object → normal **BeanOutputConverter<Movie>**.
- For multiple objects → use **BeanOutputConverter** with a **Parameterized Type Reference**.
- Ensures AI responses map directly to a **List<Movie>**.
- Returns structured JSON data that can be directly passed into Java collections.

Code Implementation:

```
@GetMapping("/moviesList")
public List<Movie> getMovieList(@RequestParam String name) {

    BeanOutputConverter<Movie> opCon = new BeanOutputConverter<Movie>(Movie.class);

    List<Movie> movies = chatClient.prompt()
        .user(u -> u.text("Top 5 movies of {name}").param("name", name))
        .call()
        .entity(new BeanOutputConverter<List<Movie>>( new ParameterizedTypeReference<List<Movie>>() {}));

    return movies;
}
```

Explanation:

- **Prompt** → Top 5 movies of {name}.
- **BeanOutputConverter with ParameterizedTypeReference** → Tells Spring AI to expect a list of **Movie** objects.
- **AI Response** → JSON array of movies, each with fields (movieName, leadActor, director, year).
- **Conversion** → Maps directly into **List<Movie>**.
- **Return Value** → Ready-to-use structured list of objects.

Example Output:

- **Request:** /moviesList?name=Tom Hanks
- **Response (JSON):**

```
[
  {
    "movieName": "Forrest Gump",
    "leadActor": "Tom Hanks",
    "director": "Robert Zemeckis",
    "year": 1994
  },
  {
    "movieName": "Cast Away",
    "leadActor": "Tom Hanks",
    "director": "Robert Zemeckis",
    "year": 2000
  }
]
```

Key Points:

- For lists, use `ParameterizedTypeReference` with **BeanOutputConverter**.
- **Works the same way as single bean conversion but returns multiple objects.**
- Useful for scenarios like lists of movies, products, books, etc.
- Provides consistent, structured data from AI to application.

Summary:

- **Bean Output Converter with List** allows converting AI output into **List<T>**.
- Requires **ParameterizedTypeReference** for type safety.
- Returns structured JSON mapped to Java objects
- Makes AI integration seamless for multiple records.